

7.26 게임 분석 배운 점

이번 분석에서는 이전의 게임 분석에서와 달리 내가 서버에게 어떤 정보를 보내는 지가 아니라 내가 서버로부터 어떤 정보를 받는지에 대해서 배웠다.

이를 위해서 먼저 ida에서 마찬가지로 exe파일을 실행시키고 이번에는 recv함수가 호출이 되는 부분으로 가보면

```
IDA View-A | Pseudocode-A | Hex View-1 | Local Types | Imports
36  v3 = recv(g_sock, &Buffer[v2], 0x2000 - v1, 0);
37  if ( v3 <= 0 )
38      return 0;
39  v4 = v1 + v3;
40  v1 = v4;
41  if ( v4 <= 5 )
42  {
43      v2 = v4;
44      memmove_0(Buffer, Buffer, v4);
45      continue;
46  }
47  offset = 0;
48  do
49  {
50      while ( 1 )
51      {
52          SIGNATURE = &Buffer[offset];
53          if ( *SIGNATURE == 'GG' )           // GG와 비교
54              break;
55          ++offset;
56          v1 = v4 - offset;
57          if ( v4 - offset <= 5 )
58              goto LABEL_12;
59      }
60  }
61  000010B9 rx_thread:54 (7FF70FB01AB9)
```

이렇게 내가 어떤 정보를 서버로부터 받는지가 대략적으로 나와 있다. 우선 첫번째로 받은 바이트 정보가 GG와 같은지를 확인하고 만약에 같다면 while loop을 탈출해서 packet id정보를 받는 듯 했고

```
63  packet_id = SIGNATURE[3];
64  v9 = &Buffer[offset + 6];
65  if ( packet_id == 2 )                     // 벽 좌표 수신
66                                          //
67  {
68      v10 = Buffer[offset + 11];
69      v11 = *v9;
70      EnterCriticalSection(&g_lock);
```

```

87     else
88     {
89         if ( packet_id == 5 )                // 플레이어 좌표 수신
90         {
91             v14 = Buffer[offset + 10];
92             EnterCriticalSection(&g_lock);
93             player_count = 32;
94             if ( v14 <= 0x20u )
95                 player_count = v14;
96             v16 = player_count;

```

packet id가 2인지 아니면 5인지에 따라서 정해진 동작을 하는 거 같아 동적 분석을 키고 실제로 recv한 값을 출력해보니 packet id가 2인 경우에는 벽 좌표를 수신한다는 점과 packet id가 5인 경우엔 플레이어 좌표를 수신한다는 점을 확인할 수 있었다.

그 다음에 나의 관심사는 플레이어의 좌표 나 뿐만 아니라 다른 사람의 위치 또한 파악하는 것이 목적이었기 때문에 우선 packet id가 5인 경우에 대해서 어떤 정보를 서버가 보내는 지에 대해서 초점을 맞추고 분석을 하기 시작했다.

```

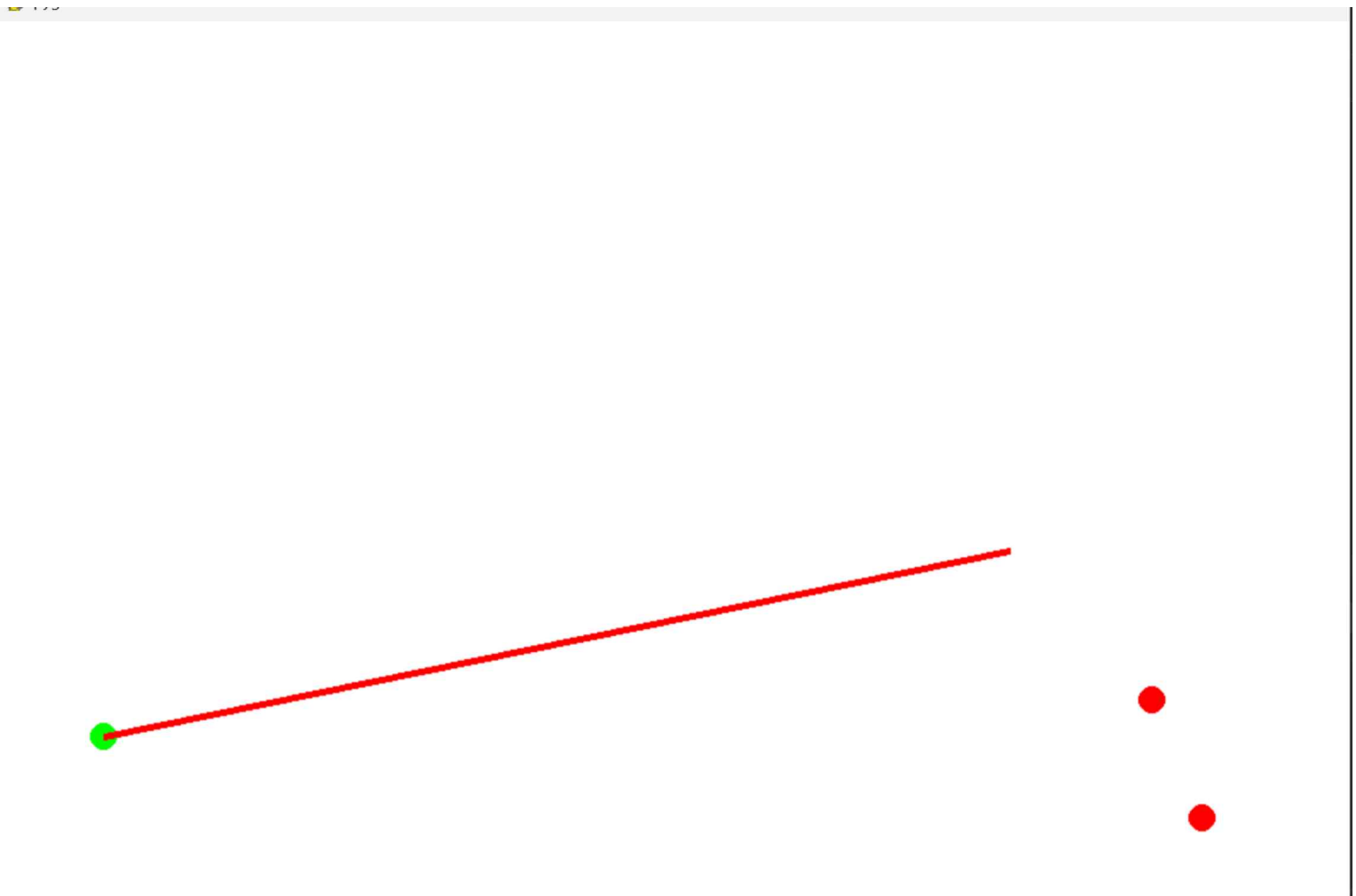
89     if ( packet_id == 5 )                // 플레이어 좌표 수신
90     {
91         v14 = Buffer[offset + 10];
92         EnterCriticalSection(&g_lock);
93         player_count = 32;
94         if ( v14 <= 0x20u )
95             player_count = v14;
96         v16 = player_count;
97         g_pl_count = player_count;
98         if ( v14 )
99         {
100             v17 = &Buffer[offset + 11];
101             players = &g_pl;
102             v19 = &v9[10 * (v16 - 1) + 15];    // 타입 변경 필요
103             do
104             {
105                 v20 = *v17;
106                 v21 = *(v17 + 1);
107                 v17 += 10;
108                 players += 6;
109                 v22 = *(v17 - 3);
110                 *(players - 24) = v20;
111                 v23 = _mm_unpacklo_epi64(
112                     _mm_unpacklo_epi32(_mm_cvtsi32_si128(v21), _mm_cvtsi32_si128(*(v17 - 7))),
113                     _mm_unpacklo_epi32(_mm_cvtsi32_si128(*(v17 - 5)), _mm_cvtsi32_si128(v22)));
114                 *(players - 1) = *(v17 - 1);
115                 *(players - 5) = v23;
116             }

```

그리고 보니 v14에 현재 전송되는 offset + 10 위치에 어떤 값을 받는 듯 했는데 이걸 xdbg64에서 디버깅을 하며 유저 수를 늘리니 증가하는 것과 유저 수와 그 값이 일치하는 것을 보고 이 값이 현재 게임에 접속한 플레이어의 수라는 점을 깨달았고 그에 따라 변수의 이름을 바꿔 주었다. 이를 통해 서버 인원 수가 32명이 최대 수용 인원이라는 점을 알 수 있었다.

그 뒤에 do while문을 보니 정확히는 모르겠지만 유저 수만큼 각 유저의 정보를 10바이트씩 저장한다는 것처럼 보여서 그 부분을 동적 분석으로 하나하나 10바이트를 이동하면서 확인한 결과 10바이트 중 첫 번째 바이트는 유저의 id번호 1,2,3,4이런 값이고 그 뒤의 2바이트는 현재 x좌표, 그 뒤의 두 바이트는 현재 y좌표라는 점을 깨달았다.

이제 이 정보를 이용해서 gpt의 도움을 받아 pygame에서 수신한 플레이어의 정보를 바탕으로 상대방의 위치를 화면에 출력하는 코드를 부탁해서 그 코드를 내가 만든 코드에 삽입하니 아래 스크린샷처럼 나는 상대방 위치를 아는 핵을 만들 수 있었다.



이제 상대방은 나의 위치를 못 보지만 나는 상대방의 위치를 파악해서 저격을 할 수 있게 된 것이다.



하지만 여기에 하나 아쉬운 점이 있다면 지금 화면에는 벽이 존재하는데 지금 코드로는 벽이 전혀 보이지 않는다는 한계가 있다. 이는 다음 분석에서 추가할 예정이다.

